

Migration guide: from `codefresh-report-image` to standalone CSDP enrichment steps

Who needs this

You need this guide if your CI currently reports images to Codefresh using **any** of these:

- The `codefresh-io/codefresh-report-image` GitHub action
- The image reporting/enrichment templates from the **GitOps Runtime** (the Argo Workflows-based enrichment)

Why: the Argo Workflows that power image enrichment inside the runtime are being **deprecated**. Image reporting keeps working — but it moves out of the runtime and into your CI pipeline, as three plain containers you run yourself. No runtime component is involved anymore.

The result in Codefresh is the same: your images appear in the [Images dashboard](#) with Git, Jira, and PR metadata attached.

What changes, in one table

Before (<code>codefresh-report-image</code>)	After (standalone steps)
One action/step	Three containers: report (mandatory) + Jira enrich (optional) + Git enrich (optional)
<code>CF_RUNTIME_NAME</code> selects a runtime	Gone. No runtime is used
<code>CF_CONTAINER_REGISTRY_INTEGRATION</code> names a Codefresh integration	You pass registry credentials directly (e.g. <code>DOCKER_USERNAME</code> / <code>DOCKER_PASSWORD</code>)
<code>CF_ISSUE_TRACKING_INTEGRATION</code> names a Codefresh integration	You pass Jira credentials directly (<code>JIRA_HOST</code> , <code>JIRA_EMAIL</code> , <code>JIRA_API_TOKEN</code>)
<code>CF_IMAGE</code>	<code>IMAGE_URI</code> (step 1) / <code>IMAGE</code> (step 2) / <code>IMAGE_SHA</code> (step 3) — same value, different names
<code>CF_GIT_BRANCH</code>	<code>GIT_BRANCH</code> (step 1) / <code>BRANCH</code> (step 3)
<code>CF_JIRA_MESSAGE</code>	<code>MESSAGE</code> (step 2)
<code>CF_JIRA_PROJECT_PREFIX</code>	<code>JIRA_PROJECT_PREFIX</code> (step 2)
<code>CF_GITHUB_TOKEN</code>	<code>GITHUB_TOKEN</code> (step 3)
<code>CF_API_KEY</code>	<code>CF_API_KEY</code> — same key works on all three steps

The three steps

#	Step	Container image	Entrypoint	Required?
1	Report image	<code>quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-report-image-info:main</code>	<code>node /usr/src/app/index.js (workdir /usr/src/app)</code>	Yes — always first
2	Jira enrichment	<code>quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-image-enricher-jira-info:main</code>	<code>node /app/src/index.js (workdir /app/)</code>	Optional, after step 1
3	Git/PR enrichment	<code>quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-image-enricher-git-info:main</code>	<code>node /app/src/index.js (workdir /app/)</code>	Optional, after step 1

Step 1 **creates** the image entity in Codefresh. Steps 2 and 3 **annotate** that entity, so they must run after step 1 finishes. Steps 2 and 3 don't depend on each other and may run in parallel. Configuration is 100% environment variables — there are no CLI arguments.

Before you start — collect these credentials

Credential	Where to get it	Used by
Codefresh API key	https://g.codefresh.io/user/settings → API Keys → Generate	All 3 steps (<code>CF_API_KEY</code>)
Registry read credentials	Your registry (e.g. Docker Hub username + token)	Step 1
Jira API token + account email	Atlassian account → Security → API tokens	Step 2
Git provider token	GitHub: the built-in job token usually suffices	Step 3

Store all of these as **secrets** in your CI system. Never hardcode them.

The 6 golden rules

Break any of these and you get failures that are hard to diagnose. Read them twice.

1. **The image URI must be byte-identical in all three steps.** Full URI including registry and tag, e.g. `docker.io/myuser/my-app:1.2.3`. If step 2 or 3 gets even a slightly different string, it annotates a nonexistent entity and your data silently goes nowhere. Define it once (one variable/expression) and reuse it.
2. **The variable NAME for the image differs per step.** Step 1: `IMAGE_URI`. Step 2: `IMAGE`. Step 3: `IMAGE_SHA`. Yes, it's inconsistent. No, you can't rename them.

3. **Always set `CF_HOST=https://g.codefresh.io` on step 1.** The code has **no default** — if you omit it, the step tries to call `undefined/2.0/api/graphql` and fails. (On-prem: use your own Codefresh URL.)
 4. **Step 2 needs a one-line patch before it runs** (see below). The image's Jira client library sends GET requests with an empty JSON body, which Atlassian's CDN (CloudFront) rejects with an opaque `403 Bad request` HTML page. Run this `sed` command in the container **before** invoking `node /app/src/index.js`:

```
sed -i "s|this.makeRequest = function (options, callback, successString)
{|this.makeRequest = function (options, callback, successString) { if (options.method
=== 'GET') { delete options.body; }|" /app/node_modules/jira-connector/index.js
```

Symptom if you forget: the step logs `body:"<!DOCTYPE HTML ... 403 ERROR ..."` right after `Looking for Issues from message ...` — and (with `FAIL_ON_NOT_FOUND=false`) still exits green, so nothing gets enriched and nothing looks broken.
 5. **GitHub Actions only: never pass the image URI through a job output if it contains a secret.** If your image path embeds a secret (e.g. `${{ secrets.DOCKERHUB_USERNAME }}`), GitHub silently drops the output (`Skip output since it may contain secret`) and the downstream step receives an **empty string**. Pass only non-secret parts (like the tag) between jobs and rebuild the full URI in each job's `env`.
 6. **JIRA_HOST is a bare hostname.** `mycompany.atlassian.net` — no `https://`, no trailing slash, no path.
-

Environment variable reference

Step 1 — report-image-info

Variable	Required	Value
IMAGE_URI	✓	Full image URI incl. registry and tag
CF_API_KEY	✓	Codefresh API key
CF_HOST	✓ (see rule 3)	https://g.codefresh.io
Registry credentials	✓ one set	Docker Hub: DOCKER_USERNAME + DOCKER_PASSWORD · ECR: AWS_ACCESS_KEY + AWS_SECRET_KEY + AWS_REGION · GCR: GCR_KEY_FILE_PATH · other: USERNAME + PASSWORD + DOMAIN
GIT_BRANCH , GIT_REVISION , GIT_COMMIT_MESSAGE , GIT_COMMIT_URL , GIT_SENDER_LOGIN	optional	Git metadata shown on the image
WORKFLOW_NAME , WORKFLOW_URL , LOGS_URL	optional	Links back to your CI run

Step 2 — image-enricher-jira-info

Variable	Required	Value
IMAGE	✓	Exactly the same URI as step 1's IMAGE_URI
CF_API_KEY	✓	Same Codefresh API key
MESSAGE	✓	Text to scan for an issue key — branch name or commit message
JIRA_PROJECT_PREFIX	✓	Your Jira project key, e.g. CR (finds CR-1234 in MESSAGE)
JIRA_HOST	✓	e.g. mycompany.atlassian.net (bare host — rule 6)
JIRA_EMAIL	✓	The Atlassian account email the token belongs to
JIRA_API_TOKEN	✓	Atlassian API token
FAIL_ON_NOT_FOUND	optional	true = fail the step when no issue matches (default false)

Step 3 — image-enricher-git-info

Variable	Required	Value
IMAGE_SHA	✓	Exactly the same URI as step 1's IMAGE_URI
CF_API_KEY	✓	Same Codefresh API key
BRANCH	✓	The PR's source branch
REPO	✓	owner/repo-name
GITHUB_TOKEN	✓	Token with read access to the repo's PRs
GIT_SENDER_LOGIN	optional	Commit author username

Example 1 — GitHub Actions

Full working workflow: [.github/workflows/docker-ci.yaml](#) in this repo. Structure: a `build` job builds and pushes the image, then:

```

csdp-report-image-info:
  runs-on: ubuntu-latest
  needs: [build]
  container:
    image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
report-image-info:main
  env:
    IMAGE_URI: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name
}}:${{ needs.build.outputs.version }}
    CF_API_KEY: ${{ secrets.CF_API_KEY }}
    CF_HOST: https://g.codefresh.io
    GIT_BRANCH: ${{ github.head_ref }}
    GIT_REVISION: ${{ github.sha }}
    GIT_COMMIT_URL: ${{ github.server_url }}/${{ github.repository }}/commit/${{ github.sha
}}
    GIT_SENDER_LOGIN: ${{ github.actor }}
    DOCKER_USERNAME: ${{ secrets.DOCKERHUB_USERNAME }}
    DOCKER_PASSWORD: ${{ secrets.DOCKERHUB_TOKEN }}
  steps:
    - name: report image info
      working-directory: /usr/src/app
      run: node /usr/src/app/index.js

csdp-image-enricher-jira-info:
  runs-on: ubuntu-latest
  needs: [build, csdp-report-image-info]
  container:
    image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
image-enricher-jira-info:main
  env:
    IMAGE: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name
}}:${{ needs.build.outputs.version }}
    CF_API_KEY: ${{ secrets.CF_API_KEY }}
    MESSAGE: ${{ github.head_ref }}
    JIRA_PROJECT_PREFIX: 'CR'
    JIRA_HOST: ${{ secrets.JIRA_HOST }}
    JIRA_EMAIL: ${{ secrets.JIRA_EMAIL }}
    JIRA_API_TOKEN: ${{ secrets.JIRA_API_TOKEN }}
    FAIL_ON_NOT_FOUND: 'false'
  steps:
    - name: enrich image with jira info
      working-directory: /app/
      run: |

```

```

# Mandatory patch - see golden rule 4
sed -i "s|this.makeRequest = function (options, callback, successString)
{|this.makeRequest = function (options, callback, successString) { if (options.method ===
'GET') { delete options.body; }|" /app/node_modules/jira-connector/index.js
node /app/src/index.js

csdp-image-enricher-github-info:
  runs-on: ubuntu-latest
  needs: [build, csdp-report-image-info]
  container:
    image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
image-enricher-git-info:main
  env:
    IMAGE_SHA: docker.io/${{ secrets.DOCKERHUB_USERNAME }}/${{ github.event.repository.name
}}:${{ needs.build.outputs.version }}
    CF_API_KEY: ${{ secrets.CF_API_KEY }}
    BRANCH: ${{ github.head_ref }}
    REPO: ${{ github.repository }}
    GITHUB_TOKEN: ${{ github.token }}
  steps:
    - name: enrich image with github info
      working-directory: /app/
      run: node /app/src/index.js

```

GitHub Actions tips:

- Trigger on `pull_request` with `types: [closed]` and gate the build job with `if: github.event.pull_request.merged == true` to run once per merged PR while keeping PR context available (a plain `push` trigger loses `github.head_ref` and PR data).
- Remember golden rule 5 about job outputs and secrets.

Example 2 — Codefresh pipeline (classic CI)

The containers run as ordinary freestyle steps. Set the shared image URI once as a pipeline variable (`IMAGE_FULL` below) to satisfy golden rule 1.

```
version: "1.0"
stages: [build, report, enrich]

steps:
  build_image:
    title: Build & push
    type: build
    stage: build
    image_name: myuser/my-app
    tag: ${CF_SHORT_REVISION}}
    registry: docker-hub          # your registry integration for pushing

  report_image_info:
    title: Report image to Codefresh
    stage: report
    image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
report-image-info:main
    working_directory: /usr/src/app
    environment:
      - IMAGE_URI=docker.io/myuser/my-app:${CF_SHORT_REVISION}}
      - CF_API_KEY=${CF_API_KEY}}
      - CF_HOST=https://g.codefresh.io
      - GIT_BRANCH=${CF_BRANCH}}
      - GIT_REVISION=${CF_REVISION}}
      - GIT_COMMIT_MESSAGE=${CF_COMMIT_MESSAGE}}
      - GIT_COMMIT_URL=${CF_COMMIT_URL}}
      - GIT_SENDER_LOGIN=${CF_COMMIT_AUTHOR}}
      - DOCKER_USERNAME=${DOCKERHUB_USERNAME}}
      - DOCKER_PASSWORD=${DOCKERHUB_TOKEN}}
    commands:
      - node /usr/src/app/index.js

  enrich_jira:
    title: Enrich with Jira issue
    stage: enrich
    image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
image-enricher-jira-info:main
    working_directory: /app/
    environment:
      - IMAGE=docker.io/myuser/my-app:${CF_SHORT_REVISION}}
      - CF_API_KEY=${CF_API_KEY}}
      - MESSAGE=${CF_BRANCH}}
      - JIRA_PROJECT_PREFIX=CR
```



```

- JIRA_HOST=mycompany.atlassian.net
- JIRA_EMAIL=${JIRA_EMAIL}
- JIRA_API_TOKEN=${JIRA_API_TOKEN}
- FAIL_ON_NOT_FOUND=false
commands:
  # Mandatory patch - see golden rule 4
  - sed -i "s|this.makeRequest = function (options, callback, successString)
{|this.makeRequest = function (options, callback, successString) { if (options.method ===
'GET') { delete options.body; }|" /app/node_modules/jira-connector/index.js
  - node /app/src/index.js

enrich_git:
  title: Enrich with PR info
  stage: enrich
  image: quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-0.0.6-images-
image-enricher-git-info:main
  working_directory: /app/
  environment:
    - IMAGE_SHA=docker.io/myuser/my-app:${CF_SHORT_REVISION}
    - CF_API_KEY=${CF_API_KEY}
    - BRANCH=${CF_BRANCH}
    - REPO=${CF_REPO_OWNER}/${CF_REPO_NAME}
    - GITHUB_TOKEN=${GITHUB_TOKEN}
  commands:
    - node /app/src/index.js

```

Store `CF_API_KEY`, `DOCKERHUB_TOKEN`, `JIRA_API_TOKEN`, `GITHUB_TOKEN` etc. as encrypted pipeline/project variables.

Example 3 — Jenkins (declarative pipeline)

Run each step inside its container with `docker.image(...).inside`. The `--entrypoint` override isn't needed — the images' default command is irrelevant because we invoke `node` explicitly.

```

pipeline {
    agent any

    environment {
        IMAGE_FULL = "docker.io/myuser/my-app:${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build & push') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable:
'DH_USER', passwordVariable: 'DH_PASS')]) {
                    sh '''
                        echo "$DH_PASS" | docker login -u "$DH_USER" --password-stdin
                        docker build -t "$IMAGE_FULL" .
                        docker push "$IMAGE_FULL"
                    '''
                }
            }
        }

        stage('Report image to Codefresh') {
            steps {
                withCredentials([
                    string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
                    usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'DOCKER_USERNAME',
passwordVariable: 'DOCKER_PASSWORD')
                ]) {
                    script {
                        docker.image('quay.io/codefreshplugins/argo-hub-workflows-codefresh-csdp-versions-
0.0.6-images-report-image-info:main').inside {
                            sh '''
                                export IMAGE_URI="$IMAGE_FULL"
                                export CF_HOST="https://g.codefresh.io"
                                export GIT_BRANCH="$BRANCH_NAME"
                                export GIT_REVISION="$GIT_COMMIT"
                                cd /usr/src/app && node index.js
                            '''
                        }
                    }
                }
            }
        }
    }
}

```

```

stage('Enrich - Jira') {
  steps {
    withCredentials([
      string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
      string(credentialsId: 'jira-api-token', variable: 'JIRA_API_TOKEN')
    ]) {
      script {
        docker.image('quay.io/codecapsule/argocd-workflows-codefresh-csdp-versions-0.0.6-images-image-enricher-jira-info:main').inside {
          sh '''
            export IMAGE="$IMAGE_FULL"
            export MESSAGE="$BRANCH_NAME"
            export JIRA_PROJECT_PREFIX="CR"
            export JIRA_HOST="mycompany.atlassian.net"
            export JIRA_EMAIL="jira-bot@mycompany.com"
            export FAIL_ON_NOT_FOUND="false"
            # Mandatory patch - see golden rule 4
            sed -i "s|this.makeRequest = function (options, callback, successString)
{|this.makeRequest = function (options, callback, successString) { if (options.method ===
'GET') { delete options.body; }|" /app/node_modules/jira-connector/index.js
            cd /app && node src/index.js
          '''
        }
      }
    }
  }
}

stage('Enrich - Git/PR') {
  steps {
    withCredentials([
      string(credentialsId: 'cf-api-key', variable: 'CF_API_KEY'),
      string(credentialsId: 'github-token', variable: 'GITHUB_TOKEN')
    ]) {
      script {
        docker.image('quay.io/codecapsule/argocd-workflows-codefresh-csdp-versions-0.0.6-images-image-enricher-git-info:main').inside {
          sh '''
            export IMAGE_SHA="$IMAGE_FULL"
            export BRANCH="$BRANCH_NAME"
            export REPO="myorg/my-app"
            cd /app && node src/index.js
          '''
        }
      }
    }
  }
}

```

```
    }  
  }  
}  
}
```

How to verify it worked

1. **Step 1 logs** should contain `REPORT_IMAGE_V2: binaryQuery response:` followed by a JSON object echoing your image name. That means the entity was created.
2. **Step 2 logs** should end with `Codefresh assign issue <KEY> to your image <uri>`.
3. **Step 3 logs** should contain `saveAnnotation` with your PR number and URL.
4. Open <https://g.codefresh.io/2.0/images> — your image should show the Git commit, the Jira issue, and the PR.

Troubleshooting

Symptom	Cause	Fix
Error: IMAGE_URI is required parameter	The env var arrived empty	Check how you pass the URI between jobs — in GitHub Actions see golden rule 5
Step 1 fails with a URL like undefined/2.0/api/graphql	CF_HOST not set	Set CF_HOST=https://g.codefresh.io (rule 3)
Jira step logs body:"<!DOCTYPE HTML ... 403 ERROR"	Missing the sed patch	Apply the patch from golden rule 4 — the credentials are probably fine
Jira step logs Issues weren't found	MESSAGE doesn't contain <PREFIX>—<number>	Pass a branch name / commit message containing the issue key, e.g. CR-1234-my-fix
The image you are trying to enrich ... does not exist	Image URI mismatch between steps	Make the URI byte-identical everywhere (rule 1)
Registry credentials is required parameter	No registry creds given to step 1	Provide one credential set (see step 1 table)
401 / auth errors from Codefresh	Bad or wrong-account CF_API_KEY	Regenerate at https://g.codefresh.io/user/settings
Enrichment "succeeds" but nothing appears on the image	Jira/Git step targeted a different URI, or Jira step silently hit the 403 above	Check rules 1 and 4; read the step logs, not just the exit code

Reference

- Working example workflow: [.github/workflows/docker-ci.yaml](#)
- Upstream step docs: [report-image-info](#) · [image-enricher-jira-info](#) · [image-enricher-git-info](#)

Note: the upstream docs describe `*_SECRET` / `*_SECRET_KEY` variants for registry credentials. Those are for Kubernetes secrets in Argo Workflows — **not applicable** when running the containers in CI. Pass the values directly as environment variables.